Planejamento de Testes - API Compass UOL ServeRest

PLANO DE TESTES DE API

- PLANO DE TESTES DE API
  - 1. Introdução
  - 2. Identificação do Projeto
  - 3. Objetivo dos Testes
  - 4. Escopo dos Testes
    - 4.1 Em Escopo (In-Scope)
    - 4.2 Fora do Escopo (Out-of-Scope)
  - 5. Riscos e Prioridades
  - 6. Estratégia de Testes
  - 7. Técnicas e Abordagens de Teste
    - 7.1 Testes Funcionais de API
    - 7.2 Testes de Contrato (API Contract Testing)
    - 7.3 Testes Exploratórios de API (SBET)
  - 8. Mapeamento dos Endpoints
    - 8.1 Autenticação
    - 8.2 Usuários
    - 8.3 Produtos
    - 8.4 Carrinhos
  - 9. Cenário de Testes (Heurísticas)
    - 9.1 Cenários Positivos
    - 9.2 Cenários Negativos
    - 9.3 Validação de Contrato da API
  - 10. Exemplos de Requisições (Modelo)
  - 10.1 🔐Login
    - 10.2 POST /login
  - 10.2 👥 Usuários
    - 10.2.1 GET /usuarios
    - 10.2.2 POST /usuarios
    - 10.2.3 GET /usuarios/{_id}
    - 10.2.4 PUT /usuarios/{_id}
    - 10.2.5 DELETE /usuarios/{_id}
  - 10.3 📦Produtos
    - 10.3.1 GET /produtos
    - 10.3.2 POST /produtos
    - 10.3.3 GET /produtos/{_id}
    - 10.3.4 PUT /produtos/{_id}
    - 10.3.5 DELETE /produtos/{_id}
  - 10.4 🛒 Carrinhos
    - 10.4.1 GET /carrinhos
    - 10.4.2 POST /carrinhos
    - 10.4.3 GET /carrinhos/{_id}
    - 10.4.4 DELETE /carrinhos/concluir-compra
    - 10.4.5 DELETE /carrinhos/cancelar-compra
  - 11. Ambiente e Ferramentas
  - 12. Papéis e Responsabilidades
  - 13. Cronograma de Testes
  - 14. Gestão de Defeitos (Bugs)
  - Cada defeito deverá conter, no mínimo:
  - 15. Critérios de Entrada e Saída
    - 15.1 Critérios de Entrada (Entry Criteria)
    - 15.2 Critérios de Saída (Exit Criteria)
  - 16. Aprovação do Plano

---

1. Introdução

Este documento descreve o planejamento de testes da API REST **ServeRest**, uma aplicação que simula uma loja virtual com funcionalidades de usuários, autenticação, produtos e carrinhos de compra.

O plano tem como objetivo validar a qualidade, confiabilidade e aderência da API às regras de negócio documentadas no Swagger oficial, conforme atividade prática proposta pela **Compass UOL**.

---

2. Identificação do Projeto

Item	Descrição
Projeto	Planejamento de Testes - API ServeRest
Sistema Alvo	ServeRest API
Tipo	API Rest
Base URL	https://compassuol.serverest.dev
Versão da API	2.29.7
Autor	Arlen Freitas Silva
Norma da Referencia	Baseado na ISO/IEC/IEEE 29119
Data	02/02/2026

---

3. Objetivo dos Testes

Os objetivos deste plano são:

- Validar os principais fluxos de negócio da API (login, usuários, produtos e carrinhos)
- Garantir que os endpoints respondam conforme contratos definidos
- Identificar falhas funcionais, de validação e de autorização
- Avaliar cenários positivos, negativos e de borda
- Demonstrar domínio em testes de API REST

---

4. Escopo dos Testes

4.1 Em Escopo (In-Scope)

IN SCOPE

- Autenticação de usuários (/login)
- CRUD de usuários (/usuarios)
- CRUD de produtos (/produtos)
- Gestão de carrinhos (/carrinhos)
- Validação de contratos (status code, schema, mensagens)
- Regras de autorização (admin x usuário comum)

4.2 Fora do Escopo (Out-of-Scope)

OUT OF SCOPE

- Testes de Performance
- Testes de carga e stress
- Testes de segurança profunda (Pentest)
- Front-end da aplicação

---

5. Riscos e Prioridades

Riscos	Impacto	Probabilidade	Mitigação
Falha de autenticação	CRÍTICO	MÉDIA	Priorizar testes funcionais e negativos no endpoint de login, validando geração, expiração e invalidação do token, além de tentativas de acesso sem autenticação em rotas protegidas.
Criação de produto sem permissão	ALTO	ALTA	Executar testes de autorização baseados em perfil, garantindo que usuários não administradores não consigam realizar operações de escrita no catálogo de produtos.
Inconsistência de estoque	CRÍTICO	MÉDIA	Aplicar testes negativos e de regra de negócio simulando operações que excedam o saldo disponível, assegurando bloqueio da transação e preservação da

			integridade dos dados.
Exclusão indevida de usuário	ALTO	BAIXA	Validar regras de integridade antes da exclusão, testando cenários onde o usuário possui dependências ativas, como carrinhos ou vínculos no sistema.
Retorno fora do contrato	MÉDIO	MÉDIA	Realizar validações de contrato verificando códigos HTTP, estrutura do payload e tipagem dos campos conforme a documentação da API.

6. Estratégia de Testes

A estratégia de testes adotada para a **API ServeRest** tem como objetivo validar o correto funcionamento dos endpoints REST, garantindo que as operações estejam aderentes aos requisitos funcionais, regras de negócio e padrões de comunicação **HTTP**.

Os testes serão realizados com foco em validações funcionais, estruturais e de contrato da API, utilizando requisições **HTTP** e análise das respostas retornadas, incluindo códigos de status, headers e corpo da resposta em formato JSON.

Serão aplicados testes positivos e negativos, considerando entradas válidas, inválidas e limites, bem como a verificação de autenticação, autorização e dependência entre endpoints.

A execução dos testes será manual, utilizando ferramentas de apoio para envio das requisições e análise das respostas, garantindo rastreabilidade entre cenários, resultados e evidências.

---

7. Técnicas e Abordagens de Teste

As técnicas e abordagens de teste adotadas neste plano foram definidas considerando as características da **API ServeRest**, que opera exclusivamente via requisições **HTTP**, sem interface gráfica, e possui dependência de autenticação, autorização, regras de negócio e contratos de comunicação bem definidos.

O objetivo da aplicação dessas técnicas é garantir que a **API** responda corretamente a diferentes tipos de entradas, respeite as regras funcionais e de segurança, e mantenha consistência nas respostas conforme a documentação

oficial.

7.1 Testes Funcionais de API

Os testes funcionais têm como foco validar o comportamento dos endpoints da **API** em relação aos requisitos esperados. Essa técnica verifica se cada operação executa corretamente sua função, considerando entradas válidas e inválidas, bem como o impacto das regras de negócio sobre as respostas retornadas.

Serão avaliados, entre outros aspectos:

- Funcionamento correto das operações de criação, consulta, atualização e exclusão de recursos;
- Respostas adequadas para entradas válidas e inválidas;
- Aplicação correta das regras de autenticação e autorização;
- Comportamento esperado em fluxos dependentes entre endpoints.

7.2 Testes de Contrato (API Contract Testing)

Os testes de contrato têm como objetivo assegurar que a **API** esteja aderente ao contrato definido em sua documentação **(Swagger)**, garantindo previsibilidade e confiabilidade para sistemas consumidores.

Essa abordagem valida:

- Códigos de status HTTP retornados conforme o tipo de operação;
- Estrutura do corpo da resposta em formato JSON;
- Presença de campos obrigatórios;
- Tipagem correta dos dados retornados;
- Consistência das mensagens de erro.

Essa técnica é essencial para evitar que mudanças no comportamento da API quebrem integrações existentes.

7.3 Testes Exploratórios de API (SBET)

Os testes exploratórios serão conduzidos de forma orientada a sessões, utilizando a técnica de **Session-Based Exploratory Testing (SBET)**, com foco em identificar comportamentos inesperados, falhas de validação e cenários não explicitamente documentados.

Essa abordagem será aplicada principalmente em áreas de maior **risco**, como autenticação, controle de estoque, permissões de acesso e fluxos de carrinho, permitindo explorar combinações de entradas e sequências de chamadas que possam revelar inconsistências ou vulnerabilidades.

---

8. Mapeamento dos Endpoints

8.1 Autenticação

Componente	Endpoint	Descrição
Login	POST /login	Realizar Login

8.2 Usuários

Componente	Endpoint	Descrição

Usuários	GET /usuarios	Listar usuários cadastrados
Usuários	POST /usuarios	Cadastrar usuário
Usuários	GET /usuarios/{_id}	Buscar usuário por ID
Usuários	PUT /usuarios/{_id}	Editar usuário
Usuários	DELETE /usuarios/{_id}	Excluir usuário

8.3 Produtos

Componente	Endpoint	Descrição
Produtos	GET /produtos	Listar produtos cadastrados
Produtos	POST /produtos	Cadastrar produto
Produtos	GET /produtos/{_id}	Buscar produto por ID
Produtos	PUT /produtos/{_id}	Editar produto
Produtos	DELETE /produtos/{_id}	Excluir produto

8.4 Carrinhos

Componente	Endpoint	Descrição
Carrinhos	GET /carrinhos	Listar carrinhos cadastrados
Carrinhos	POST /carrinhos	Cadastrar carrinho
Carrinhos	GET /carrinhos/{_id}	Buscar carrinho por ID
Carrinhos	DELETE /carrinhos/concluir-compra	Excluir carrinho
Carrinhos	DELETE /carrinhos/cancelar-compra	Excluir carrinho e retornar produtos para estoque

---

9. Cenário de Testes (Heurísticas)

Abordagem Heurística Utilizada

Os cenários de teste definidos nesta seção foram elaborados com base nas seguintes heurísticas de teste:

- Testes Funcionais Baseados em Regras de Negócio
- Testes Negativos baseados em Error Guessing
- Testes de Autorização e Controle de Acesso
- Testes de Fluxo de Negócio (End-to-End de API)

Essas heurísticas permitem validar o comportamento funcional da API, seus mecanismos de segurança e a consistência dos fluxos críticos de negócio.

9.1 Cenários Positivos

Login com credenciais válidas

Validar que a API permita a autenticação de usuários com e-mail e senha corretos, retornando um token de acesso válido que possibilite o consumo das rotas protegidas.

Cadastro de usuário com dados válidos

Verificar se a API permite o cadastro de usuários comuns e administradores quando os dados obrigatórios são informados corretamente, garantindo a persistência adequada das informações.

Criação de produto por usuário administrador

Confirmar que usuários com perfil administrador conseguem cadastrar produtos no catálogo com sucesso, respeitando as regras de autorização da API.

Criação de carrinho com produtos válidos

Validar o fluxo de criação de carrinho, assegurando que os produtos informados sejam reservados corretamente e que o cálculo do valor total seja realizado de forma automática.

9.2 Cenários Negativos

Login com credenciais inválidas

Garantir que a API bloqueie tentativas de autenticação com dados incorretos, retornando mensagem de erro e código de status apropriado.

Cadastro de usuário com e-mail já existente

Validar a restrição de unicidade do campo e-mail, impedindo o cadastro de usuários duplicados.

Criação de produto sem autorização

Verificar se a API impede que usuários não administradores realizem operações de criação, edição ou exclusão de produtos.

Exclusão de usuário com carrinho ativo

Testar a regra de integridade que impede a exclusão de usuários que possuem carrinhos associados.

Inconsistência de estoque

Tentar adicionar ao carrinho uma quantidade de produtos superior ao estoque disponível, validando o bloqueio da operação e a manutenção da integridade dos dados.

9.3 Validação de Contrato da API

⚠ **Códigos de status HTTP**
Verificar se a API retorna códigos de status coerentes com cada tipo de operação, como 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized e 403 Forbidden.

⚠ **Estrutura da resposta**
Validar se o corpo das respostas segue a estrutura definida na documentação da API, contendo todos os campos obrigatórios.

⚠ **Tipagem de dados**
Confirmar que os campos retornados respeitam os tipos de dados esperados, como string, integer e boolean.

⚠ **Mensagens de erro padronizadas**
Assegurar que as mensagens de erro sejam consistentes e compreensíveis, facilitando o tratamento por sistemas consumidores da API.

ℹ Os cenários descritos representam fluxos críticos e riscos relevantes da API ServeRest, não tendo como objetivo cobrir exaustivamente todos os endpoints, mas sim demonstrar a aplicação de heurísticas de teste e a validação dos comportamentos mais sensíveis do sistema.

---

10. Exemplos de Requisições (Modelo)

ℹ Esta seção apresenta exemplos ilustrativos de requisições para os principais endpoints da API, com foco na compreensão dos payloads e no comportamento esperado das respostas.

10.1 🔐Login

10.2 POST /login

ℹ **ID do Cenário:** SR-API-001
Endpoint: POST /login Realizar Login
URL Base: ServeRest
Tipo: Exemplo de Requisição – Funcional
Heurística Aplicada:

- Teste Funcional baseado em Regra de Negócio
- Controle de Acesso (Autenticação)

Headers:

- Accept: application/json
- Content-Type: application/json

Params: N/A
Payload (Body):

```
{
   "email": "fulano@qa.com",
   "password": "teste"
 }
```

Resposta 200 OK

Login realizado com sucesso

```
{
    "message": "Login realizado com sucesso",
```

```
    "authorization": "Bearer eyJhbGciOiJIUzI1NiIs..."
}
```

Possíveis Erros

401 UNAUTHORIZED

```
{
  "message": "Email e/ou senha inválidos"
}
```

10.2 👥 Usuários

10.2.1 GET /usuarios

ID do Cenário: SR-API-002

Endpoint: GET `/usuarios` Listar Usuários Cadastrados

URL Base: ServeRest

Tipo: Funcional / Negativo

Heurística Aplicada:

- Teste Negativo baseado em Validação de Entrada
- Controle de Acesso (Autenticação)

Headers:

- Accept: application/json
- Authorization: Bearer [TOKEN]

Params: N/A

Payload (Body): N/A

Response Body 200 OK

Lista de usuários:

```
{
  "quantidade": 1,
  "usuarios": [
    {
      "nome": "Fulano da Silva",
      "email": "beltrano@qa.com.br",
      "password": "teste",
      "administrador": "true",
      "_id": "0uxuPY0cbmQhpEz1"
    }
  ]
}
```

Possíveis Erros

401 UNAUTHORIZED

```
{
  "message": "Token inválido ou não informado"
}
```

10.2.2 POST /usuarios

ID do Cenário: SR-API-003

Endpoint: POST `/usuarios` Cadastrar Usuários

URL Base: ServeRest

Tipo: Exemplo de Requisição – Contrato

Heurística Relacionada:

- Teste de Contrato de API
- Validação de Estrutura JSON

Headers:

- Accept: application/json
- Content-Type: application/json

Params: N/A

Payload (Body):

```
{
  "nome": "Fulano da Silva",
  "email": "beltrano@qa.com.br",
  "password": "teste",
  "administrador": "true"
}
```

Response Body 201 OK CREATED

Cadastrado com sucesso

```
{
    "message": "Cadastro realizado com sucesso",
    "_id": "IkZ8b0vt0fxfy6H4"
}
```

Possíveis Erros

400 BAD REQUEST

```
{
  "message": "Este email já está sendo usado"
}
```

10.2.3 GET /usuarios/{_id}

ID do Cenário: SR-API-004

Endpoint: GET `/usuarios/{_id}` Buscar usuário por ID

URL Base: ServeRest

Tipo: Exemplo de Requisição – Funcional

Heurística Relacionada:

- Consulta por Identificador
- Integridade de Dados

Headers:

- Accept: application/json
- Authorization: Bearer [TOKEN]

Params: _id (path param obrigatório)

Payload (Body): N/A

Response Body 200 OK

Usuário encontrado

```
{
    "nome": "Fulano da Silva",
    "email": "beltrano@qa.com.br",
    "password": "teste",
    "administrador": "true",
    "_id": "IkZ8b0vt0fxfy6H4"
}
```

Possíveis Erros

400 BAD REQUEST

```
{
  "message": "Usuário não encontrado"
}
```

10.2.4 PUT /usuarios/{_id}

ID do Cenário: SR-API-005

Endpoint: PUT `/usuarios/{_id}` Editar usuário

URL Base: ServeRest

Tipo: Exemplo de Requisição – Funcional

Heurística Relacionada:

- Atualização de Recurso
- Consistência de Dados

Headers:

- Accept: application/json
- Content-Type: application/json
- Authorization: Bearer [TOKEN]

Params: _id (path param obrigatório)

Payload (Body):

```
{
  "nome": "Fulano da Silva",
  "email": "beltrano@qa.com.br",
  "password": "teste",
  "administrador": "true"
}
```

Response Body 200 OK

Alterado com sucesso

```
{
  "message": "Registro alterado com sucesso"
}
```

Response Body 201 CREATED

```
{
    "message": "Cadastro realizado com sucesso",
    "_id": "bVR8AzE8ZyBwLzGz"
}
```

Possíveis Erros

400 BAD REQUEST :

```
{
  "message": "Este email já está sendo usado"
```

```
}
```

10.2.5 DELETE /usuarios/{_id}

ID do Cenário: SR-API-006

Endpoint: DELETE `/usuarios/{_id}` Excluir usuário

URL Base: ServeRest

Tipo: Exemplo de Requisição – Funcional

Heurística Relacionada:

- Remoção Controlada
- Regra de Integridade

Headers:

- Accept: application/json
- Authorization: Bearer [TOKEN]

Params: _id (path param obrigatório)

Payload (Body): N/A

Response Body 200 OK

Registro excluído com sucesso | Nenhum registro excluído

```
{
  "message": "Registro excluído com sucesso | Nenhum registro excluído"
}
```

Possíveis Erros

400 BAD REQUEST

```
{
  "message": "Não é permitido excluir usuário com carrinho cadastrado
  "idCarrinho": "qbMqntef4iTOwWfg"
}
```

10.3 📦Produtos

10.3.1 GET /produtos

ID do Cenário: SR-API-007

Endpoint: GET `/produtos` Listar produtos cadastrado

URL Base: ServeRest

Tipo: Exemplo de Requisição – Contrato

Heurística Aplicada:

- Teste de Contrato de API
- Validação de Estrutura JSON
- Consistência de Tipos de Dados

Headers:

- Accept: application/json
- Authorization: Bearer [TOKEN]

Params: N/A

Payload (Body): N/A

Response Body 200 OK

Lista de produtos

```
{
  "quantidade": 2,
  "produtos": [
    {
      "nome": "Logitech MX Vertical",
      "preco": 470,
      "descricao": "Mouse",
      "quantidade": 382,
      "_id": "BeeJh5lz3k6kSIzA"
    },
    {
      "nome": "Samsung 60 polegadas",
      "preco": 5240,
      "descricao": "TV",
      "quantidade": 49977,
      "_id": "K6leHdftCeOJj8BJ"
    },
  ]
}
```

Possíveis Erros

400 BAD REQUEST

```
{
    "message": "Já existe produto com esse nome"
}
```

10.3.2 POST /produtos

ID do Cenário: SR-API-008

Endpoint: POST `/produtos` **Cadastrar produtos**

URL Base: ServeRest

Tipo: Exemplo de Requisição – Funcional / Positivo

Heurística Aplicada:

- Teste Funcional baseado em Regra de Negócio
- Controle de Acesso (Autorização por Perfil)
- Persistência de Dados

Headers:

- Accept: application/json
- Content-Type: application/json
- Authorization: Bearer [TOKEN]

Params: N/A

Payload (Body):

```
{
    "nome": "Teclado Mecânico",
    "preco": 350,
    "descricao": "Teclado com switches azuis",
    "quantidade": 5
}
```

Response Body 201 CREATED

Cadastro com sucesso

```
{
    "message": "Cadastro realizado com sucesso",
    "_id": "GAS7dRJOaSVfI2fm"
}
```

Clique aqui para expandir...

400 BAD REQUEST

```
{
  "message": "Já existe produto com esse nome"
}
```

401 UNAUTHORIZED

```
{
  "message": "Token de acesso ausente, inválido, expirado ou usuário
}
```

403 FORBIDDEN

```
{
  "message": "Rota exclusiva para administradores"
}
```

10.3.3 GET /produtos/{_id}

ID do Cenário: SR-API-009

Endpoint: GET `/produtos/{_id}` Buscar Produto por ID

URL Base: ServeRest

Tipo: Exemplo de Requisição – Funcional

Heurística Aplicada:

- Consulta por Identificador
- Integridade de Dados
- Teste de Contrato de Resposta

Headers:

- Accept: application/json
- Authorization: Bearer [TOKEN]

Params: _id (path param obrigatório)

Payload (Body): N/A

Response Body 200 OK

Produto Encontrado

```
{
  "nome": "Logitech MX Vertical",
  "preco": 470,
  "descricao": "Mouse",
  "quantidade": 381,
  "_id": "BeeJh5lz3k6kSIzA"
}
```

Possíveis Erros

400 BAD REQUEST Produto não encontrado

```
{
  "message": "Produto não encontrado"
}
```

10.3.4 PUT /produtos/{_id}

ID do Cenário: SR-API-010

Endpoint: PUT `/produtos/{_id}` Editar Produto

URL Base: ServeRest

Tipo: Exemplo de Requisição – Funcional / Negativo

Heurística Aplicada:

- Controle de Acesso (Autorização)
- Consistência de Atualização
- Regra de Negócio

Headers:

- Accept: application/json
- Content-Type: application/json
- Authorization: Bearer [TOKEN]

Params: _id (path param obrigatório)

Payload (Body):

```
{
  "nome": "Logitech MX Vertical",
  "preco": 470,
  "descricao": "Mouse",
  "quantidade": 381
}
```

Response Body 200 OK

Alterado com sucesso

```
{
  "message": "Registro alterado com sucesso"
}
```

Response Body 201 CREATED

Cadastro com sucesso

```
{
  "message": "Cadastro realizado com sucesso",
  "_id": "jogfODIlXsqxNFS2"
}
```

Clique aqui para expandir...

400 BAD REQUEST

```
{
  "message": "Já existe produto com esse nome"
}
```

401 UNAUTHORIZED

```
{
  "message": "Token de acesso ausente, inválido, expirado ou usuário
}
```

403 FORBIDDEN

```
{
  "message": "Rota exclusiva para administradores"
```

```
}
```

10.3.5 DELETE /produtos/{_id}

ID do Cenário: SR-API-011

Endpoint: DELETE `/produtos/{_id}` Excluir Produto

URL Base: ServeRest

Tipo: Exemplo de Requisição – Funcional / Negativo

Heurística Aplicada:

- Remoção Controlada de Recurso
- Integridade Referencial
- Controle de Autorização

Headers:

- Accept: application/json
- Authorization: Bearer [TOKEN]

Params: _id (path param obrigatório)

Payload (Body): N/A

Response Body 200 OK

```
{
  "message": "Registro excluído com sucesso | Nenhum registro excluído"
}
```

Clique aqui para expandir...

400 BAD REQUEST

```
{
  "message": "Não é permitido excluir produto que faz parte de carrin
  "idCarrinho": [
    "qbMqntef4iTOwWfg, yILJY1eaAUC6hyRc"
  ]
}
```

401 UNAUTHORIZED

```
{
  "message": "Token de acesso ausente, inválido, expirado ou usuário
}
```

403 FORBIDDEN

```
{
  "message": "Rota exclusiva para administradores"
}
```

10.4 🛒 Carrinhos

10.4.1 GET /carrinhos

ID do Cenário: SR-API-012

Endpoint: GET `/carrinhos` **Listar Carrinhos Cadastrados**

URL Base: ServeRest

Tipo: Exemplo de Requisição – Contrato

Heurística Aplicada:

- Teste de Contrato de API
- Validação de Estrutura JSON
- Consistência de Dados Agregados

Headers:

- Accpet: application/json
- CAuthorization: Bearer [TOKEN]

Params: N/A

Payload (Body): N/A

Response Body 200 OK

Lista de carrinhos

```
{
    "quantidade": 2,
    "produtos": [
        {
            "nome": "Logitech MX Vertical",
            "preco": 470,
            "descricao": "Mouse",
            "quantidade": 382,
            "_id": "BeeJh5lz3k6kSIzA"
        },
        {
            "nome": "Samsung 60 polegadas",
            "preco": 5240,
            "descricao": "TV",
            "quantidade": 49977,
            "_id": "K6leHdftCeOJj8BJ"
        }
    ]
}
```

Possíveis Erros

401 UNAUTHORIZED

```
{
  "message": "Token inválido ou não informado"
}
```

10.4.2 POST /carrinhos

ID do Cenário: SR-API-013

Endpoint: POST `/carrinhos` Cadastrar Carrinhos

URL Base: ServeRest

Tipo: Exemplo de Requisição – Funcional / Positivo

Heurística Aplicada:

- Teste Funcional baseado em Regra de Negócio
- Manipulação de Estado (State Transition)
- Validação de Consistência de Estoque

Headers:

- Accept: application/json
- Content-Type: application/json
- Authorization: Bearer [TOKEN]

Params: N/A

Payload (Body):

```
{
  "produtos": [
    {
      "idProduto": "BeeJh5lz3k6kSIzA",
      "quantidade": 1
    },
    {
      "idProduto": "YaeJ455lz3k6kSIzA",
      "quantidade": 3
    }
  ]
}
```

Response Body 201 CREATED

Cadastro com sucesso

```
{
  "message": "Cadastro realizado com sucesso",
  "_id": "jogf0DIlXsqxNFS2"
}
```

Possíveis Erros

400 BAD REQUEST

```
{
  "message": "Não é permitido possuir produto duplicado | Não é permi
}
```

401 UNAUTHORIZED

```
{
  "message": "Token de acesso ausente, inválido, expirado ou usuário
}
```

10.4.3 GET /carrinhos/{_id}

ID do Cenário: SR-API-014

Endpoint: GET `/carrinhos/{_id}` **Buscar Carrinho por ID**

URL Base: ServeRest

Relacionamento com Cenário: 9.3 Validação de Contrato

Tipo: Exemplo de Requisição – Funcional

Heurística Aplicada:

- Consulta por Identificador
- Integridade de Estado
- Teste de Contrato de Resposta

Headers:

- Accept: application/json
- Authorization: Bearer [TOKEN]

Params: _id (path params obrigatório)

Payload (Body): N/A

Response Body 200 OK

Carrinho encontrado:

```
{
  "produtos": [
    {
      "idProduto": "BeeJh5lz3k6kSIzA",
      "quantidade": 2,
      "precoUnitario": 470
    },
    {
      "idProduto": "K6leHdftCeOJj8BJ",
      "quantidade": 1,
      "precoUnitario": 5240
    }
  ],
  "precoTotal": 6180,
  "quantidadeTotal": 3,
  "idUsuario": "0uxuPY0cbmQhpEz1",
  "_id": "qbMqntef4iTOwWfg"
}
```

Possíveis Erros

400 BAD REQUEST

```
{
  "message": "Carrinho não encontrado"
}
```

10.4.4 DELETE /carrinhos/concluir-compra

ID do Cenário: SR-API-015

Endpoint: DELETE `/carrinhos/concluir-compra` **Excluir Carrinho**

URL Base: ServeRest

Tipo: Exemplo de Requisição – Funcional / Positivo

Heurística Aplicada:

- Transação de Negócio
- Consistência de Dados Pós-Ação
- Persistência de Alteração de Estado

Headers:

- Accept: application/json
- Authorization: Bearer [TOKEN]

Params: N/A

Payload (Body): N/A

Response Body 200 OK

Registro excluído com sucesso | Não foi encontrado carrinho para esse usuário

```
{
  "message": "Registro excluído com sucesso | Não foi encontrado carrinho para esse usuário"
}
```

Possíveis Erros

401 UNAUTHORIZED

```
{
  "message": "Token de acesso ausente, inválido, expirado ou usuário
}
```

10.4.5 DELETE /carrinhos/cancelar-compra

ID do Cenário: SR-API-016

Endpoint: DELETE `/carrinhos/cancelar-compra` Excluir carrinho e retomar produtos para estoque

URL Base: ServeRest

Tipo: Exemplo de Requisição – Funcional / Negativo

Heurística Aplicada:

- Rollback de Transação
- Reversão de Estado
- Integridade de Estoque

Headers:

- Accept: application/json
- Authorization: Bearer [TOKEN]

Params: N/A

Payload (Body): N/A

Response Body 200 OK

Registro excluído com sucesso | Não foi encontrado carrinho para esse usuário

```
{
  "message": "Registro excluído com sucesso | Não foi encontrado carrinho para esse usuário"
}
```

Possíveis Erros

401 UNAUTHORIZED

```
{
  "message": "Token de acesso ausente, inválido, expirado ou usuário
}
```

---

11. Ambiente e Ferramentas

- **Ambiente:** Produção pública (ServeRest)
- **Ferramenta de Testes:** Postman
- **Documentação:** Swagger
- **Gestão:** Jira com plugin **QAlity** (para gestão de casos de teste e execução)
- **Evidências:** Prints e JSONs (se necessário)

---

12. Papéis e Responsabilidades

Este plano de testes será executado em contexto acadêmico, considerando uma estrutura simplificada de papéis.

- **Analista de QA / Testador:**
  Responsável pelo planejamento, execução dos testes de API, registro e acompanhamento de defeitos, bem como pela validação dos resultados obtidos.
- **Instrutor / Avaliador:**
  Responsável pela validação do plano de testes, orientação técnica e avaliação da qualidade do planejamento e dos artefatos produzidos.
- **API ServeRest (Sistema em Teste):**
  Responsável por disponibilizar os endpoints e comportamentos definidos na documentação para fins de teste.

---

13. Cronograma de Testes

O cronograma abaixo representa uma estimativa simplificada para execução das atividades de teste, considerando o escopo da atividade.

Período	Atividade
1 dia	**Planejamento dos testes**
1 dia	**Mapeamento dos endpoints e análise de riscos**
1 dia	**Execução dos testes funcionais, negativos e de contrato**
1 dia	**Registro e análise de defeitos**
1 dia	**Revisão e entrega do plano de testes**

O cronograma poderá ser ajustado conforme a complexidade encontrada durante a execução dos testes.

---

14. Gestão de Defeitos (Bugs)

A gestão de defeitos será realizada de forma manual, com registro estruturado das falhas identificadas durante a execução dos testes.

Cada defeito deverá conter, no mínimo:

1. Identificação do endpoint afetado
2. Descrição clara do problema
3. Passos para reprodução
4. Resultado esperado
5. Resultado obtido
6. Severidade e prioridade:
   Crítica: Impede a compra/fluxo principal.
   Alta: Funcionalidade importante falha, mas há contorno.
   Média/Baixa: Problemas cosméticos ou de usabilidade menor.
7. Evidências (logs ou prints, quando aplicável)

Os defeitos identificados servirão como insumo para avaliação da qualidade da API e não implicam correção, uma vez que a atividade possui caráter acadêmico.

---

15. Critérios de Entrada e Saída

15.1 Critérios de Entrada (Entry Criteria)

ENTRADA

– Ambiente de testes disponível e acessível.
– API operacional e respondendo às requisições HTTP.
– Ferramenta de testes configurada para envio das requisições.

15.2 Critérios de Saída (Exit Criteria)

SAÍDA

– Todos os cenários planejados executados.
– Resultados devidamente registrados e evidenciados.
– Defeitos identificados documentados conforme padrão definido.

---

16. Aprovação do Plano

Nome	Cargo	Data	Assinatura
Arlen Freitas	Dev Automação de Testes	02/02/2026	*Arlen Freitas Silva*